

PGR208 – Android Programming

Hjemmeeksamen – Kandidatnummer 67

Rapport

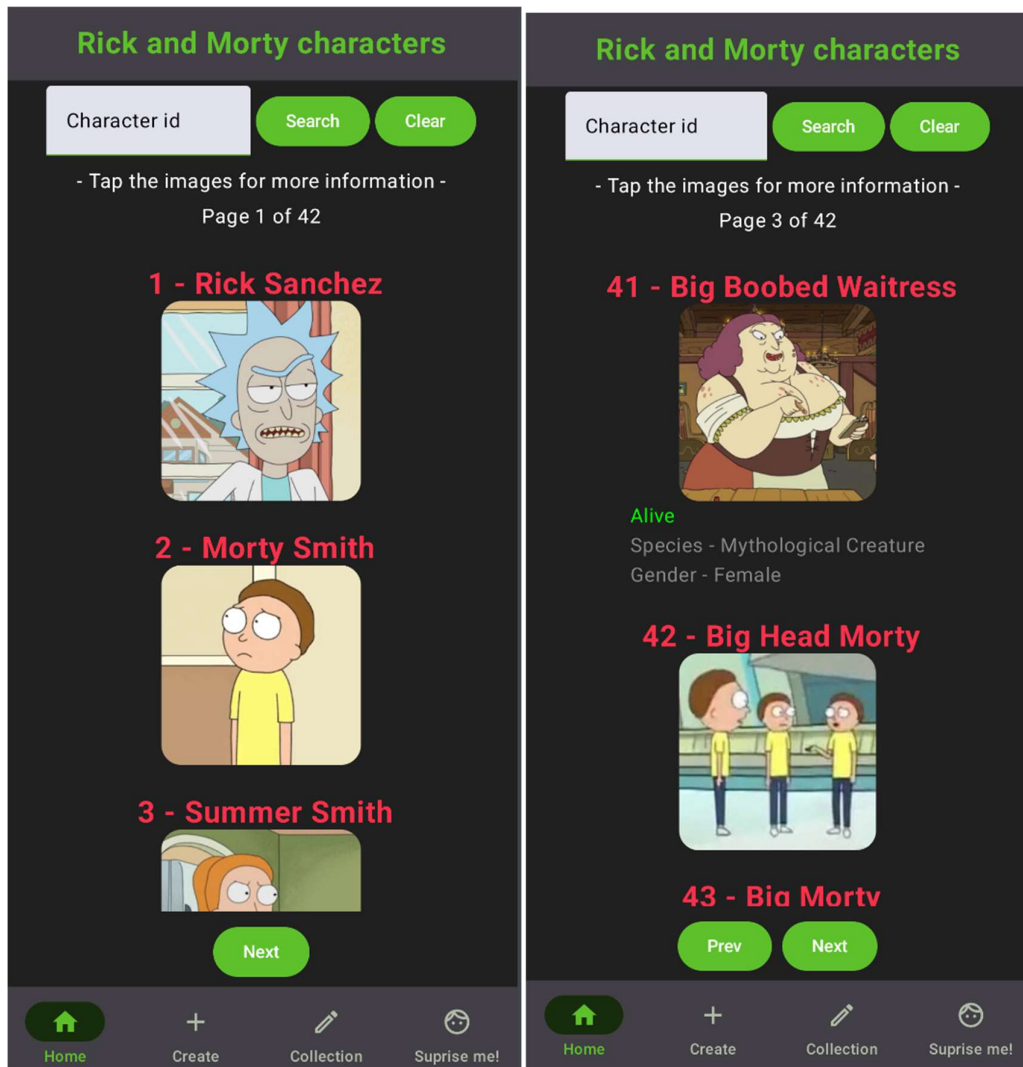
Oversikt over funksjonalitet

Side	Funksjonalitet	Beskrivelse
Home	Søke etter karakterer	Søkefunksjon etter karakterer sin ID
Home	Viser karakterer fra APIet	Viser alle karakterer i APIet ved hjelp av knapper.
Create	Lage karakterer	Brukeren skal lage karakterer som blir lagret i en database.
Collection	Se karakterer	Viser alle karakterer som er lagret i databasen
Collection	Slette karakterer	Mulighet til å slette karakterer ved hjelp av en knapp.
Suprise me	Få en karakter	Viser tilfeldig karakter ved trykk på knappen.
Suprise me	Få en ny karakter	Viser en ny tilfeldig karakter ved trykk på knappen.

Skjermbilder av appen med beskrivelse

Side 1 – Home

Siden viser 20 karakterer om gangen, med navigasjonsknapper nederst. Tekstfeltet øverst brukes til å søke etter karakterer basert på ID. «Clear»-knappen fjerner søket, slik at alle karakterer vises igjen. Alle bildene kan trykkes på for å vise mer informasjon.



Side 2 – Create

Siden lar deg opprette en egen karakter som blir lagt inn i databasen. Brukeren skriver inn de diverse verdiene i tekstfeltene og trykker på knappen for å lagre.

Create a character!

Name

Status

Gender

Species

Add Character

Home Create Collection Suprise me!

Side 3 – Collection

Siden viser alle karakterene brukeren har laget. Ved å trykke på den røde søppelbøtten vil karakteren bli slettet fra databasen.

Characters that you have made

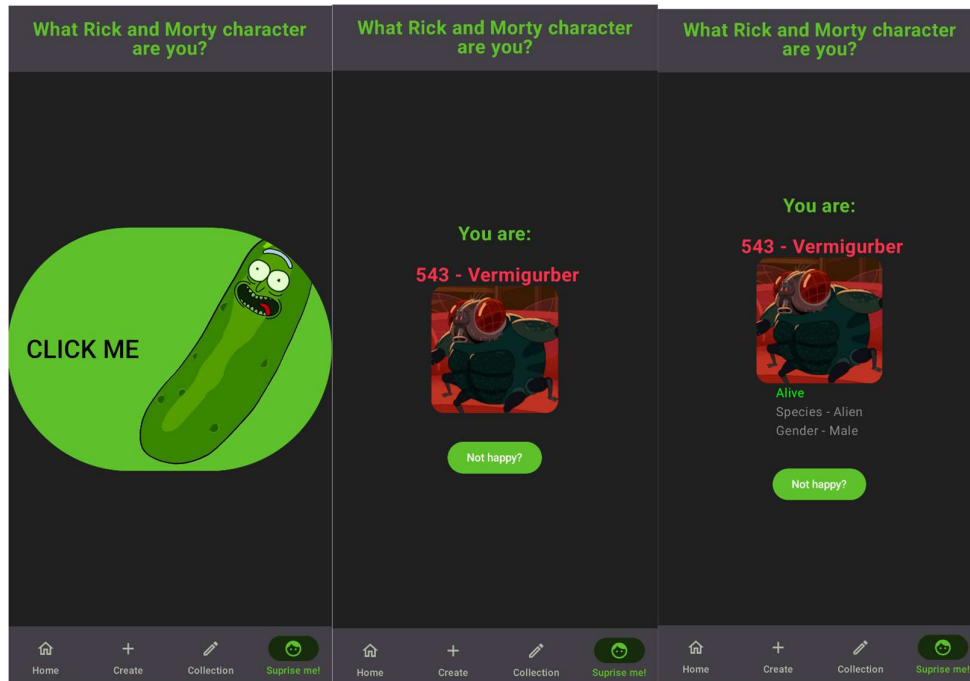
Pepper
Status: Dead
Species: Dog-pickle
Gender: Male

Salt
Status: Alive
Species: Alien-pickle
Gender: Female

Home Create Collection Suprise me!

Side 4 – Surprise me

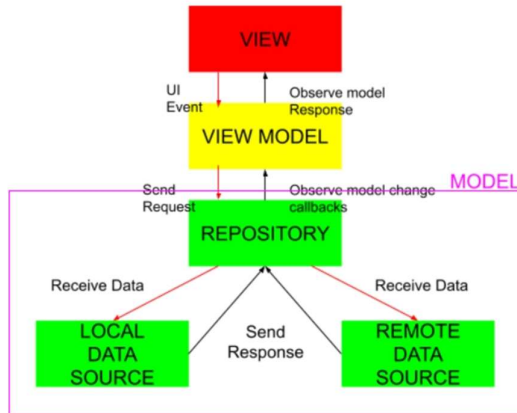
Siden viser først en stor knapp, som videre vil vise en tilfeldig karakter fra APIet. Deretter kommer en ny knapp for å prøve igjen. Bildet kan klikkes på for mer informasjon.



Beskrivelse av hovedteknikker

Jeg har brukt hovedteknikker som MVVM, Retrofit og Room.

MVVM arkitekturen skiller kode med forskjellig funksjonalitet og består av delene View, ViewModel og Data Layer(Model). ViewModel fungerer som et mellomledd som håndterer data mellom screens (View) og APIet/Database (Data Layer). Dette gjør koden min mer oversiktlig og enklere å vedlikeholde.



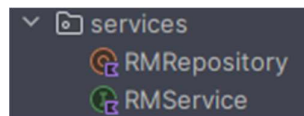
Figur 1- MVVM

Retrofit-biblioteket inneholder teknikker for å håndtere http-kall, og konverterer API-responsen til JSON-objekter ved hjelp av Gson. Jeg har benyttet Retrofit til å kommunisere med APIet, og opprettet RMService for å utføre kallene som henter karakter og diverse informasjon som trengs for å utføre handlingene i appen. RMRepository oppretter en klient og inneholder metoder som videre brukes i ViewModels.

```
interface RMService {

    @GET("character")
    suspend fun getCharactersByPage(
        @Query("page") page: Int = 1
    ): Response<CharacterList>

    @GET("character/{id}/")
    suspend fun getCharacterById(
        @Path("id") id: Int
    ) : Response<Character>
}
```



Room-biblioteket inneholder teknikker for å lagre data og kommunisere med SQLite-databasen. Denne databasen er en del av appen i mobiltelefonen, og inneholder bare data fra appen. CRUD blir brukt til å kommunisere med databasen i et Dao Interface, men i min applikasjon er det bare blitt brukt CRD, da jeg ikke har en funksjon for å oppdatere databasen inne i CharacterDao. CharacterDatabase oppretter og definerer SQLite databasen. CharacterRepository initialiserer databasen og inneholder metoder som er mellomledet mellom ViewModels og databasen.

```
@Dao
interface CharacterDao {

    @Query("SELECT * FROM MadeCharacter")
    suspend fun getMadeCharacters() : List<MadeCharacter>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertMadeCharacter(madeCharacter: MadeCharacter) : Long

    @Delete
    suspend fun deleteMadeCharacter(madeCharacter: MadeCharacter) : Int
}
```

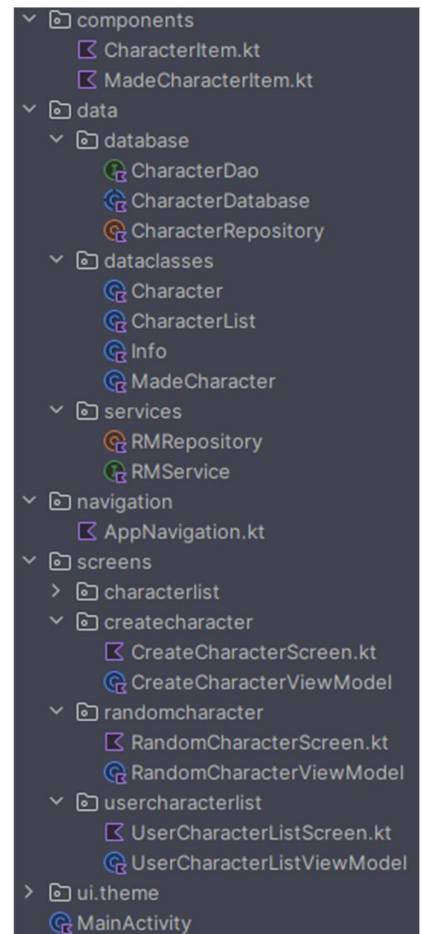


Kvalitet og struktur

Prosjektet er delt logisk opp i mapper.

- Components inneholder elementer som kan gjenbrukes i prosjektet.
- Data inneholder kode for å hente data.
- Dataclasses til klasser som definerer data.
- Navigation inneholder navigasjonsbaren.
- Screens inneholder mapper som har screen med tilhørende ViewModel.

Denne strukturen har gjort prosjektet mye enklere å jobbe med, da det var enkelt å navigere meg frem til mapper, da kode med forskjellig ansvar var adskilt. Variabler og funksjoner fått logiske navn som beskriver hva de gjør, som for eksempel `getCharactersById`, som gjorde det enkelt å finne frem til riktig metode. Repeterende farger eller visuelle modifikasjoner har egne variabler for å gjøre koden enklere å vedlikeholde. Alle exceptions er skrevet i samme stil for å ha for bedre kontroll over feil, og er da lettere å finne i LogCat.



```
fun getCharacterById(id: Int) {  
    viewModelScope.launch {  
        try {  
            _character.value = RMRepository.getCharacterById(id)  
        } catch (e: Exception) {  
            Log.d(  
                tag: "Error - getCharactersById",  
                msg: "Problems getting characters from the API - ${e.message}"  
            )  
        }  
    }  
}
```

Kilder:

Figur 1 - Naukri.com. «Android MVVM (Model-View-ViewModel) Architecture – bilde». Tilgjengelig 19. november 2024. <https://www.naukri.com/code360/library/android-mvvm-model-view-viewmodel-architecture>.

Bilde brukt inne i applikasjonen:

Pickle Rick. «Picke Rick-bilde.». Tilgjengelig 19. november 2024. <https://www.pinclipart.com/maxpin/iTxbTom/>